

MongoDB Replication

MongoDB replication is one of the **most important concepts** in database administration and production systems.

- Explain **WHY replication is needed**
- Explain **HOW replica sets work internally**
- Show the **workflow**
- Explain the **real-time industry usage**
- Demonstrate the **architecture visually**
- Connect concepts to **failure handling and high availability**
- Monitoring the performance**

1. What is Replication in MongoDB?

Definition

Replication means:

Maintaining multiple copies of the same database data across different MongoDB servers.

MongoDB uses **Replica Sets** for replication.

2. Why Replication is Needed?

Without replication:

- If the database server crashes → application stops
- If hardware fails → data loss
- If maintenance is required → downtime
- If traffic increases → performance issues

Replication solves these problems.

3. Benefits of Replication

A. High Availability

If one server fails:

- another server automatically becomes active
- application continues working

Example

Banking application should work 24/7.

If the main DB server crashes:

another server takes over automatically.

B. Data Redundancy

Data exists in multiple servers.

If one server loses data:

other copies still exist.

C. Disaster Recovery

Useful during:

hardware failures
power outages
cloud server crashes

D. Read Scalability

Secondary servers can handle read operations.

This reduces load on primary server.

E. Automatic Failover

MongoDB automatically elects a new primary.

No manual intervention required.

4. What is a Replica Set?

Definition

A Replica Set is:

A group of MongoDB servers maintaining the same data.

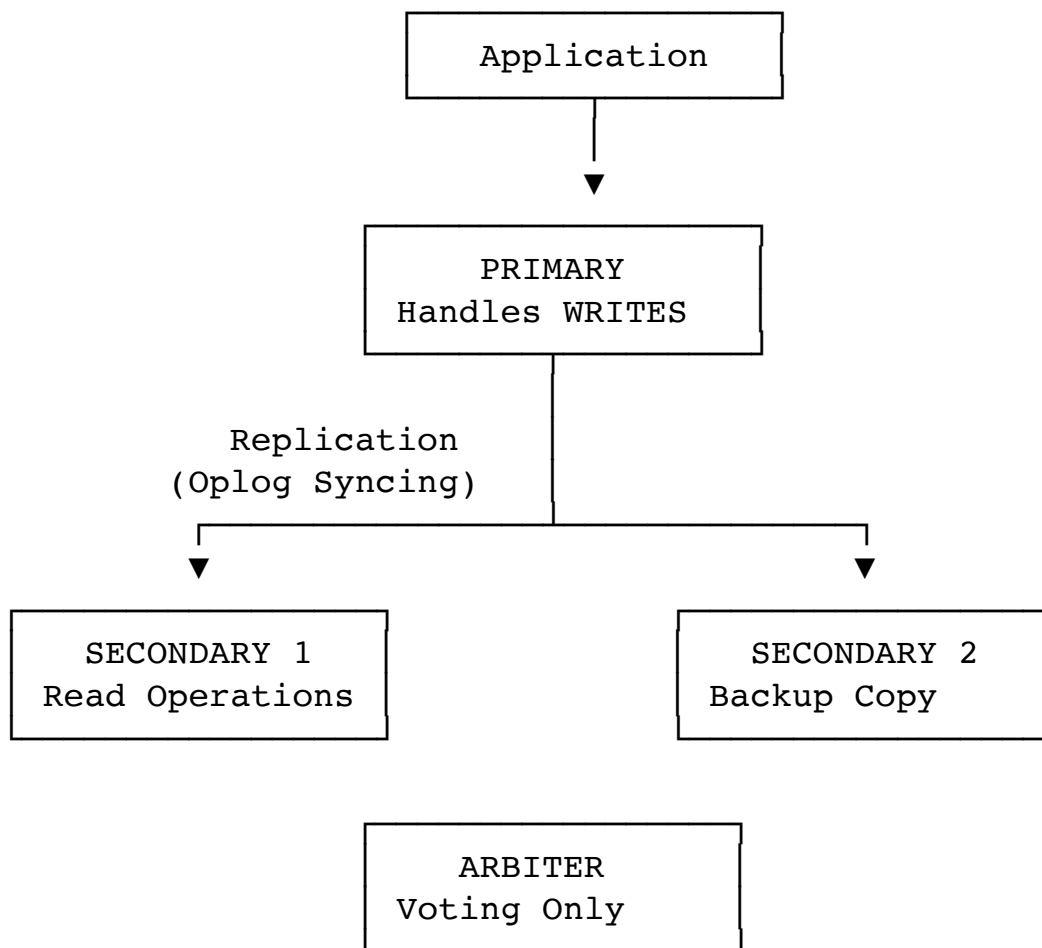
5. Replica Set Architecture

Typically consists of:

Member Type	Purpose
Primary	Handles writes
Secondary	Copies data from primary
Arbiter	Participates in voting only

6. Replica Set Architecture Diagram

Basic Architecture



7. Purpose of Each Member

PRIMARY

Responsibilities

accepts insert/update/delete

- maintains latest data
- records operations in oplog

Example

```
db.students.insertOne({  
  name: "John",  
  mark: 95  
})
```

Write goes only to PRIMARY.

SECONDARY

Responsibilities

- copies data from primary
- can serve read operations
- used for backup

Real-Time Usage

Analytics dashboards often read from secondaries.

ARBITER

Responsibilities

- does NOT store data
- participates in voting
- helps in election process

Useful when:

- limited hardware available

8. How Replication Works Internally

MongoDB uses:

OPLOG (Operations Log)

What is Oplog?

Oplog is a special collection:

```
local.oplog.rs
```

It stores all write operations.

```
db.student.insertOne({id:1,name:"Alice",age:12}) - write
```

```
db.student.find() - read
```

9. Replication Workflow

Step-by-Step Workflow

STEP 1:

Application sends INSERT request



STEP 2:

PRIMARY writes data



STEP 3:

PRIMARY records operation in OPLOG



STEP 4:

SECONDARIES continuously read OPLOG



STEP 5:

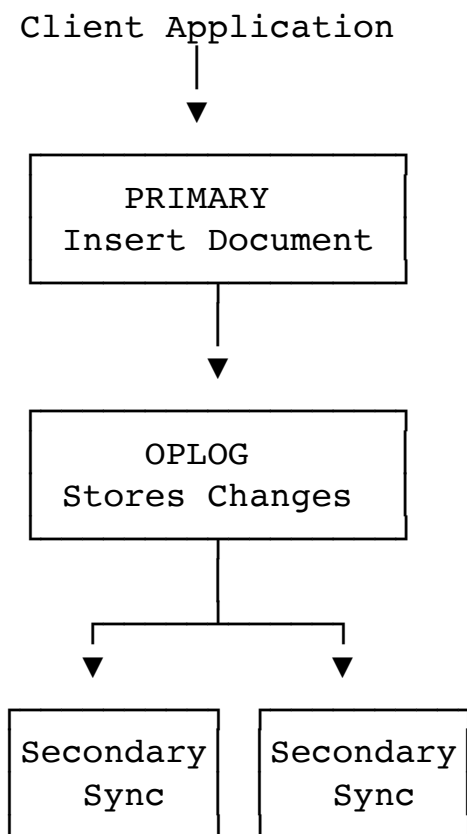
SECONDARIES apply same operation



STEP 6:

All servers now contain same data

10. Workflow Diagram



11. Primary-Secondary Replication Model

MongoDB follows:

Primary-Secondary Architecture

Important Rule

WRITES → PRIMARY ONLY

READS →

Primary

OR

Secondary (optional)

12. Example Scenario

E-Commerce Website

PRIMARY

Handles:

order placement

payment updates

inventory updates

SECONDARY

Handles:

reports

analytics

dashboards

13. Automatic Failover

This is the most exciting concept for beginners.

Scenario

Suppose:

Primary server crashes

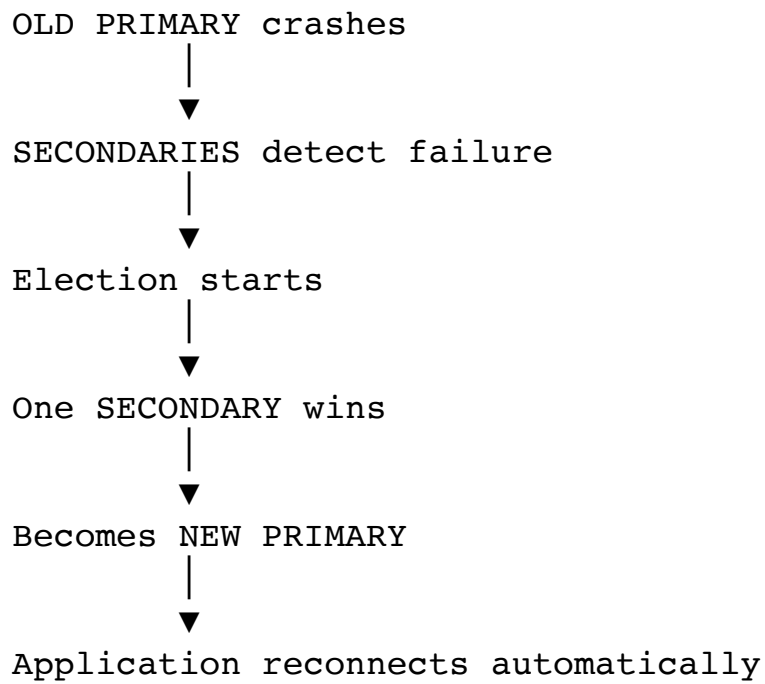
What happens?

Election Process

Remaining servers conduct election.

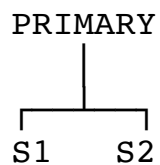
One secondary becomes new primary.

14. Failover Workflow

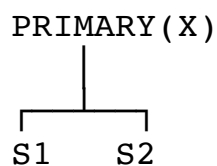


15. Failover Architecture Diagram

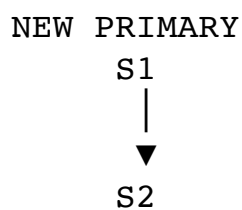
BEFORE FAILURE



AFTER FAILURE



Election Happens



16. Configuring Replica Sets

Now explain practical configuration.

17. Step 1 – Start MongoDB Instances

Example:

```
mongod --replSet rs0 --port 27017 --dbpath /data/db1
mongod --replSet rs0 --port 27018 --dbpath /data/db2
mongod --replSet rs0 --port 27019 --dbpath /data/db3
```

18. Step 2 – Connect to Mongo Shell

```
mongosh --port 27017
```

19. Step 3 – Initialize Replica Set

```
rs.initiate()
```

20. Step 4 – Check Status

```
rs.status()
```

21. Add Members to Replica Set

Syntax

```
rs.add("localhost:27018")
```

```
rs.add("localhost:27019")
```

22. Adding Arbiter

```
rs.addArb("localhost:27020")
```

23. Removing Members

Syntax

```
rs.remove("localhost:27019")
```

24. Monitoring Replication Status

Important admin commands.

A. Check Replica Status

```
rs.status()
```

Shows:

- primary
- secondary
- health
- sync status

B. Replication Info

```
rs.printReplicationInfo()
```

Shows oplog information.

C. Secondary Replication Delay

```
rs.printSecondaryReplicationInfo()
```

Shows lag time.

25. Understanding Replication Lag

What is Replication Lag?

Delay between:

- primary update
- secondary synchronization

Example

```
Primary updated at 10:00:00  
Secondary updated at 10:00:03
```

Replication Lag = 3 seconds

26. Real-Time Industry Usage

Banking Systems

Purpose:

- zero downtime
- high availability
- disaster recovery

E-Commerce Platforms

Purpose:

- continuous order processing
- read scaling
- analytics on secondaries

Examples:

- product catalogs

recommendations
reports

Social Media Applications

Purpose:

millions of read requests
continuous availability

Secondaries handle:

profile views
feed loading

Healthcare Systems

Purpose:

patient data safety
backup copies
failover protection

OTT Platforms

Examples:

streaming history
recommendation systems
user activity logs

Create Project Structure

Inside folder:

```
mongodb-replication-demo/  
├── scripts/  
│   ├── setupReplica.js  
│   ├── insertData.js  
│   ├── checkData.js  
│   └── failoverDemo.js  
└── data/  
    ├── db1/  
    └── db2/
```

| |— db3/

9. Open VS Code Terminal

In VS Code:

Terminal → New Terminal

10. Start MongoDB Servers

Open 3 separate terminals in VS Code.

TERMINAL 1

Run:

```
mongod --replSet rs0 --port 27017 --dbpath ./data/db1
```

TERMINAL 2

Run:

```
mongod --replSet rs0 --port 27018 --dbpath ./data/db2
```

TERMINAL 3

Run:

```
mongod --replSet rs0 --port 27019 --dbpath ./data/db3
```

Trainer Explanation

Explain:

We are running 3 MongoDB servers on same machine using different ports.

11. Create Replica Set Initialization Script

Create file:

scripts/setupReplica.js

Add:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "localhost:27017" },
    { _id: 1, host: "localhost:27018" },
    { _id: 2, host: "localhost:27019" }
  ]
});

print("Replica Set Initialized");
```

12. Execute Replica Set Script

Open NEW terminal.

Run:

```
mongosh --port 27017 scripts/setupReplica.js
```

13. Verify Replica Set

Connect:

```
mongosh --port 27017
```

Run:

```
rs.status()
```

Expected Result

```
PRIMARY  
SECONDARY  
SECONDARY
```

DEMO 1 — Insert Data into Primary

14. Create Insert Script

Create:

```
scripts/insertData.js
```

Add:

```
use("trainingDB");  
  
db.students.insertMany([  
  {  
    name: "John",  
    marks: 90  
  },  
  {  
    name: "Sara",  
    marks: 95  
  }  
]);  
  
print("Data Inserted Successfully");
```

15. Execute Insert Script

Run:

```
mongosh --port 27017 scripts/insertData.js
```

16. Verify Data in Primary

Connect:

```
mongosh --port 27017
```

Run:

```
use("trainingDB")
```

```
db.students.find().pretty()
```

Teaching Point

Explain:

Primary handles all write operations.

DEMO 2 — Show Data in Secondary

17. Connect to Secondary

Open new terminal.

```
mongosh --port 27018
```

18. Enable Reads on Secondary

Run:

```
rs.slaveOk()
```

OR

```
db.getMongo().setReadPref("secondary")
```

19. Read Replicated Data

Run:

```
use("trainingDB")
```

```
db.students.find().pretty()
```

Teaching Point

Explain:

Data replicated automatically from PRIMARY using OPLOG.

DEMO 3 — Stop Primary Server

20. Identify Primary

Run:

```
rs.status()
```

Suppose:

```
27017 → PRIMARY
```

21. Stop Primary

Go to terminal running 27017.

Press:

CTRL + C

Teaching Point

Ask students:

Will database stop completely?

Then explain:

MongoDB election process.

DEMO 4 — Show Election and New Primary

22. Connect to Secondary

```
mongosh --port 27018
```

23. Check Status

Run:

```
rs.status()
```

Expected Output

```
27018 → PRIMARY  
27019 → SECONDARY
```

27017 → DOWN

24. Verify New Primary

Run:

```
db.hello()
```

Look for:

```
isWritablePrimary: true
```

25. Insert New Data into New Primary

Run:

```
use("trainingDB")
```

```
db.students.insertOne({  
  name: "David",  
  marks: 99  
})
```

26. Verify Data

```
db.students.find().pretty()
```

Teaching Point

Explain:

Application continues even after server failure.
This is HIGH AVAILABILITY.

DEMO 5 — Restart Old Primary

27. Restart Server

Go back to terminal 1.

Run again:

```
mongod --replSet rs0 --port 27017 --dbpath ./data/db1
```

28. Wait Few Seconds

MongoDB automatically:

- rejoins replica set
- syncs data
- becomes secondary

29. Connect to Restarted Node

```
mongosh --port 27017
```

30. Enable Read

```
rs.slaveOk()
```

31. Verify Synced Data

```
use("trainingDB")
```

```
db.students.find().pretty()
```

Expected Output

John
Sara
David

Teaching Point

Explain:

Old primary automatically synchronized missing data from
oplog.

32. Final Replica Status

Run:

```
rs.status()
```

Expected:

Node	Role
27018	PRIMARY
27017	SECONDARY
27019	SECONDARY

33. Best Visual Teaching Workflow

During training:

Action	Show to Students
Insert	<code>find()</code> in primary
Replication	<code>find()</code> in secondary
Failure	CTRL + C
Election	<code>rs.status()</code>
Recovery	synced documents

34. Important Commands Summary

Purpose	Command
Start MongoDB	mongod
Connect Shell	mongosh
Initialize Replica	rs.initiate()
Status	rs.status()
Check Primary	db.hello()
Enable Secondary Reads	rs.slaveOk()
Stop Server	CTRL + C

35. Real-Time Industry Mapping

Classroom Demo	Real Industry Example
Insert Student	Customer Order
Replication	Backup Data Centers
Primary Failure	Cloud Server Crash
Election	Auto Failover
Recovery	Disaster Recovery